

课程笔记

ClaudeCode 源码精读 P3: prompt.ts (二)

五道口纳什 · 笔记整理

2026 年 4 月 26 日



视频作者/频道: 五道口纳什

发布日期: 2025

视频时长: 17:50

视频链接: <https://www.bilibili.com/video/BV1v9DBBtEym?p=3>

目录

1	getSimpleDoingTasks 入口与软件工程任务规则	3
1.1	问题：用户没设置 output style 时，系统提示词如何额外补一段” 软件工程任务怎么做”	3
1.2	主张：通过三目运算符按 output style 状态分支，把工程纪律以” 段落级 instructions” 挂载进来	4
1.3	机制：先理解再修改、不主动新建文件、不估时、不盲目重试、求助优先于硬撑、安全优先、不过度抽象、不留半成品	5
1.4	示例：用户说” 改成蛇形命名法” 必须改代码而非空回；安装失败别在升降级里死循环；三段相似代码优于过早抽象	7
1.5	收束：这些条目看似废话，但正对应 LLM 编码常见反模式，是” 对实习生的兜底”	7
1.6	本章小结	7
2	过早抽象、自我披露与 Output Style 的 Plugin 哲学	8
2.1	问题：为什么系统提示词要管” 代码该怎么写”	8
2.2	主张：好的智能体应该” 在对话框里完成自我披露”	8
2.3	机制：output style 不是硬编码，而是 plugin + hook 动态注入	10
2.4	示例：Learning output style 如何把 Claude 变成” 小老师”	11
2.5	收束：plugin 是被当成” 产品” 来做的	12
2.6	本章小结	13
3	Executing Actions with Care: getActions 的风险意识设计	13
3.1	问题：模型该怎么在执行动作前自己判断风险	13
3.2	主张：动作能否执行 = 可逆性 × 影响面 × 批准范围	13
3.3	机制：危险操作必须主动征求同意，一次批准不传递	14
3.4	示例：实习生类比——若不告知，他就直接 <code>rm -rf</code> 了	15
3.5	收束：spirit and letter / measure twice, cut once	15
3.6	风险矩阵：可逆性 vs. 影响面的四象限分析	16
3.7	本章小结	17
4	getTools: 专用工具为何优于原始 Bash 命令	18
4.1	问题：既然 Bash 已经够用，为什么还要 Read / Edit / Grep	18
4.2	主张：专用工具不是 Bash 的” 语法糖”	19
4.2.1	维度一：用户理解审阅	20
4.2.2	维度二：可逆回滚与审计	20
4.2.3	维度三：模型准确率与 harness 可调校性	20
4.2.4	维度四：跨平台一致性	22
4.3	机制：prompt.ts 把工具集合作为” 可替换的接口”	23
4.4	本章小结	24
5	TodoWrite、REPL 聚合、并行调用与 Tone & Style 收尾	24
5.1	任务列表与并行工具调用	24
5.2	REPL 聚合工具：把原子调用降级为虚拟消息	25
5.3	工具接口的持续优化：bfs 与 ugrep	27

5.4	Tone & Style 与输出风范	27
5.5	本章小结	29
6	总结与延伸	30
6.1	讲师的实质性收尾: spirit and letter 与输出风范	30
6.2	跨章节的四条主线	30
6.3	实践启示与开放问题	31
6.4	拓展阅读	31

1 getSimpleDoingTasks 入口与软件工程任务规则

1.1 问题：用户没设置 output style 时，系统提示词如何额外补一段” 软件工程任务怎么做”

上一讲我们沿着 `prompt.ts` 自上而下读到 `getSimpleDoingTasks` 段落的入口。本讲从这里的三目运算符切入，一路读完软件工程任务规则、`/help` 自我披露、`getActions` 的风险三件套、`getTools` 的专用工具理由、`TodoWrite` 与并行调用、`REPL` 聚合工具，直到 `tone-and-style` 段落收尾。读完这一集，你会看到 Claude Code 的系统提示词是如何把” 工程纪律 + 风险意识 + 工具协议 + 输出风格” 四件事统一灌进同一个文件的。

在 `prompt.ts` 的 `getSystemPrompt()` 函数体中，`getSimpleDoingTasksSection()` 的调用被包裹在一个三目运算符（ternary）之内：

```
1 const simpleDoingTasksSection =
2   !outputStyleConfig || outputStyleConfig.keepCodingInstructions
3   ? getSimpleDoingTasksSection()
4   : '';
```

Listing 1: `getSystemPrompt()` 中的条件分支示意：当 `outputStyleConfig` 未设置或 `keepCodingInstructions` 为真时，才会注入软件工程任务规则段。

这段代码的含义非常直白：如果用户没有设置 `output style`，或者当前 `output style` 的配置中保留了 `keepCodingInstructions` 标志，那么系统提示词就会额外拼接一段由 `getSimpleDoingTasksSection()` 生成的” 软件工程任务怎么做” 的说明；否则，这一段被留空，不进入最终 `prompt`。

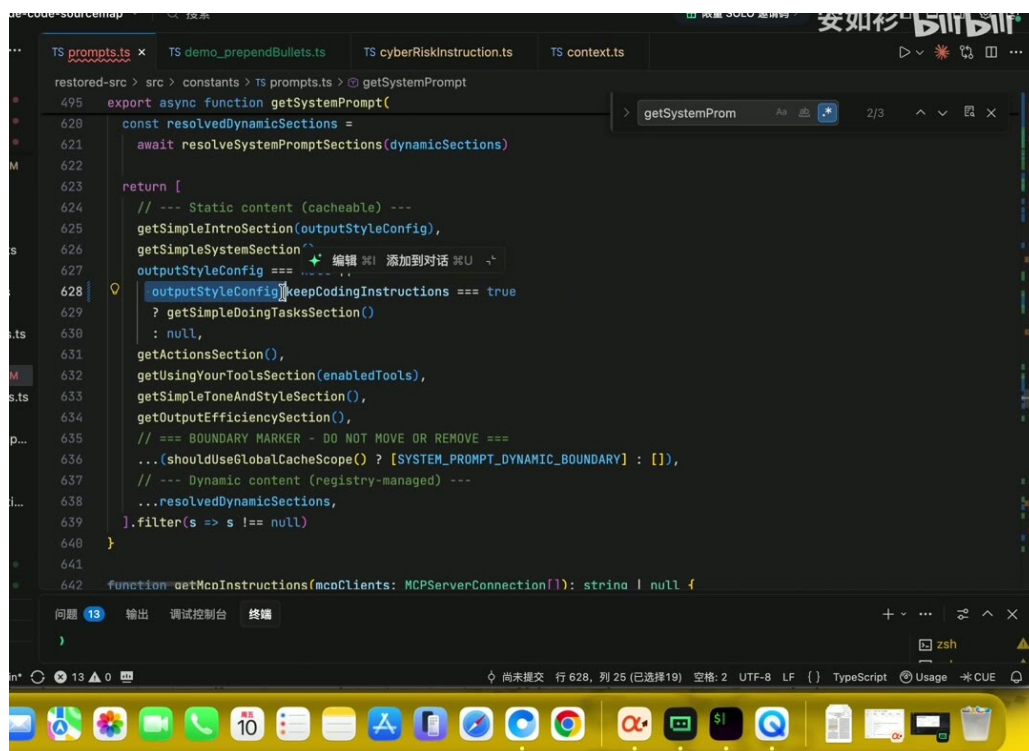


图 1: `getSystemPrompt()` 的拼装逻辑：当 `outputStyleConfig` 未设置或 `keepCodingInstructions` 为真时，才会注入软件工程任务规则段。¹

¹ 视频画面时间区间：00:00:00–00:00:10。

为什么需要这个分支? 因为 Claude Code 支持多种 output style (如 Learning 模式、Explanatory 模式), 这些风格会改变模型的行为基调。当用户切换到非编码风格时, 系统提示词不应再灌输”修复漏洞、新增功能、重构代码”等软件工程纪律, 否则会造成行为冲突。三目运算符在这里充当了一个**静态路由闸**, 按配置状态决定哪些 section 进入最终提示词。

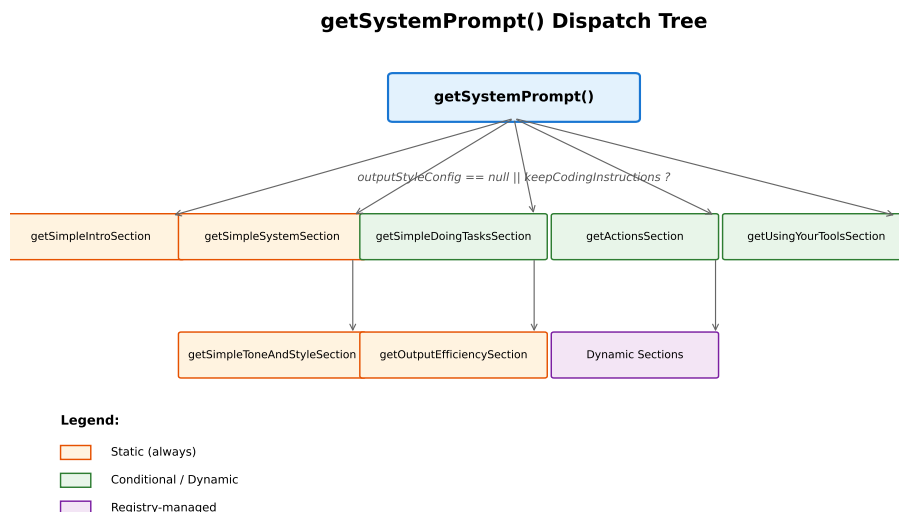


图 2: `getSystemPrompt()` 的拼装树: 条件分支决定哪些 section 被注入最终提示词。²

1.2 主张: 通过三目运算符按 output style 状态分支, 把工程纪律以”段落级 instructions”挂载进来

当条件满足时, `getSimpleDoingTasksSection()` 会返回一段结构化的 instructions。这段文本不是零散的建议, 而是围绕”如何执行软件工程任务”展开的完整段落级规则。讲师在视频中逐条展开, 指出这些规则看似”给实习生的唠叨”, 实则对应 LLM 编码最常见的反模式。

²视频画面时间区间: 00:00:00-00:00:10。

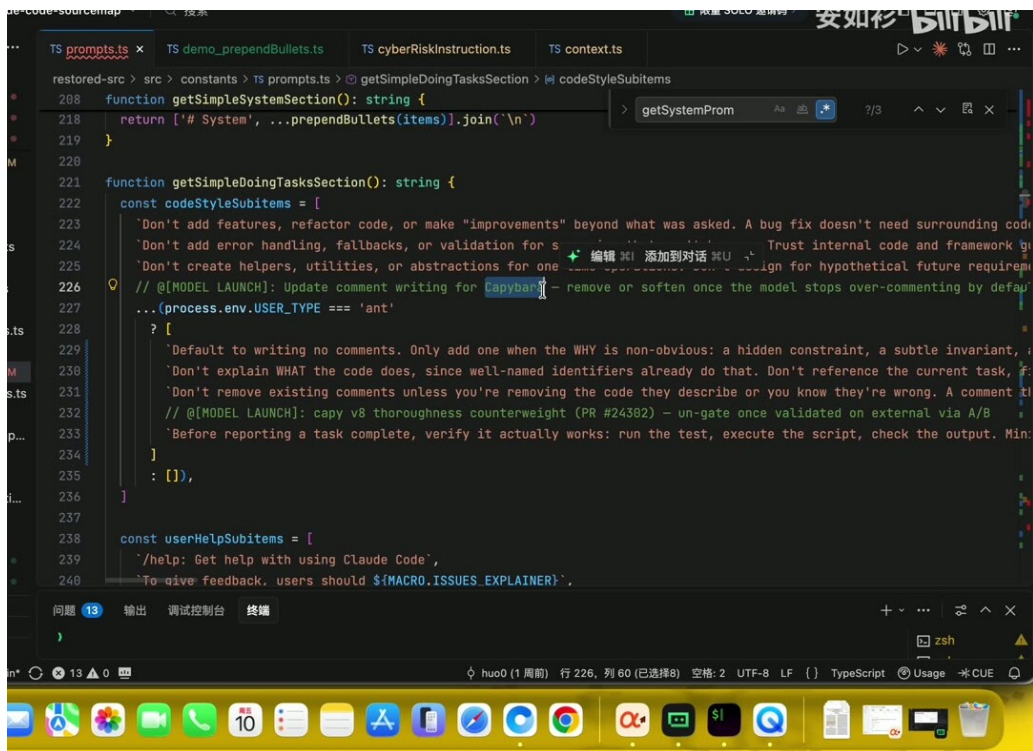


图 3: getSimpleDoingTasksSection 源码片段：从”不要过度抽象”到”/help 自我披露”的代码级实现。³

1.3 机制：先理解再修改、不主动新建文件、不估时、不盲目重试、求助优先于硬撑、安全优先、不过度抽象、不留半成品

getSimpleDoingTasksSection() 生成的规则可以归纳为八条核心纪律。下面按视频讲解顺序逐项展开。

(1) **结合工作目录理解模糊指令。**系统提示词告诉模型：遇到不明确的指令时，需要结合当前工作目录来做理解。例如，用户说”改成蛇形命名法”，模型不能只做文字回复，而要在代码中找到对应的方法并修改代码。这条规则针对的是 LLM 的一种常见幻觉——用户给了一个模糊指令，模型基于常识而非实际代码作答，结果给出的修改建议与代码现状脱节。

(2) **先阅读再修改。**系统提示词明确要求：不要对未阅读过的代码提出修改建议；在提出修改建议之前，一定要先理解。讲师在视频中调侃这”好像是废话”，但紧接着指出大模型确实可能出现”没读过代码就敢改”的幻觉。

阅

读优先于修改——*Don't suggest changes to code you haven't read.* 模型必须先通过 Read 工具或上下文中的代码片段建立对现状的准确理解，再给出修改方案。

(3) **禁止凭空新建文件，优先编辑既有文件。**除非绝对必要，不要创建新文件；通常应该编辑现有的文件，这样可以避免文件冗余，也能更高效地基于现有工作来推进。讲师提到，编码智能体特别喜欢”一个功能就是一个脚本”的散乱架构，缺乏在原有基础上改进的意识。

³视频画面时间区间：00:06:35–00:06:45。

禁

止凭空新建文件，应优先编辑既有文件以避免“一功能一脚本”的散乱架构。系统提示词把这条规则写进去，是为了防止模型在遇到需求时第一反应是“新开一个文件”，而不是在现有代码中寻找可复用的结构。

(4) **不要对任务耗时给出估算。**模型应当重点说明完成的事项，而不是所需的时间。讲师解释，LLM 其实不太了解“时间”是个什么样的东西，让它估时往往产生无意义的数字。与其给出一个不靠谱的 ETA，不如把注意力放在“做了什么”和“下一步做什么”上。

(5) **失败后不要盲目重试，陷入困境时主动向用户提问。**若某一方法失效，在更换策略之前需要先仔细分析原因、查看错误信息、检查假设并尝试修复。不要盲目做相同的操作，也不要因为一次失败就放弃可行的方法。当调查后确实陷入困境时，应该询问用户相关问题，向用户求助。

讲师在这里特别感慨系统提示词“为什么能说得这么详细”。他举了一个典型的 LLM 反模式：安装某个依赖失败，模型开始降级版本；降级版本又失败，继续降级；降到最低级还是不行，又升级到最高级——最终陷入死循环。如果系统提示词里没有“陷入困境时向用户求助”这条规则，模型就会在这个循环里硬撑，而不是停下来获取新的上下文。

失

败后不要盲目重试同一动作；陷入困境时**主动向用户提问**优先于继续硬试。向用户提问是获取新上下文的重要选项，其优先级高于无意义的版本升降级循环。

(6) **安全优先，不要站在用户要求外新增功能。**若发现自己写了不安全的代码，需要修复。安全、可靠、正确的代码是非常重要的。不要站在用户的要求外新增功能、重构代码或进行所谓的优化。这条规则约束的是 LLM 的“过度热心”——用户只要求改一个变量名，模型却顺手把整个模块重写了。

(7) **简洁代码，不要乱加注释。**系统提示词明确要求生成的代码要简洁化。讲师提到，早期 AI 写代码特别喜欢写很多注释，“就怕你看不懂”。但过多的注释不仅增加噪音，还可能在代码演进后变成误导。好的代码应该通过命名和结构自解释，而非靠注释兜底。

(8) **不要为不可能场景写错误处理、不留 TODO 半成品、不要过度抽象。**对于不可能发生的场景，不要添加错误处理、备用方案或验证逻辑。这也是 AI 的一个通病：特别喜欢写一些防御性代码，为永远不会触发的分支做处理。此外，不要为一次性的操作创建对应的工具或抽象层；不要为假设未来的需求来做设计。

三

段相似代码优于过早抽象 (rule of three / *Premature abstraction is the root of all evil*)。复杂度应贴合任务本身，不要为假设未来的需求来做设计。过早抽象是 LLM 与新手工程师最常见的反模式之一。

不

为不可能场景写错误处理、不留 TODO 半成品、不要乱加注释。这些条目看似琐碎，但正对应 LLM 编码的常见通病：防御性代码泛滥、任务未完成就标记为 done、注释冗余且易过时。

1.4 示例：用户说”改成蛇形命名法”必须改代码而非空回；安装失败别在升降级里死循环；三段相似代码优于过早抽象

上述八条纪律在视频中都有具体的讲师评论作为”现实锚点”。例如：

- **蛇形命名法示例**（字幕 00:00:46）：用户要求”将这个改为蛇形命名法（snake case）”，模型不能只做文字回复，而要在代码中找到对应的方法并修改代码。这对应规则（1）——模糊指令必须结合工作目录理解。
- **版本死循环示例**（字幕 00:03:28）：安装失败 → 降级 → 再失败 → 再降级 → 降到最低 → 升级到最高 → 回到原点。这对应规则（5）——陷入困境时应向用户提问，而非在死循环里硬撑。
- **三段相似代码示例**（字幕 00:04:25）：不要为一次性的操作创建对应的工具或抽象层；通用工具或抽象层只有在重复出现三次以上时才值得考虑。这对应规则（8）——过早抽象是 LLM 编码最常见的反模式之一。

1.5 收束：这些条目看似废话，但正对应 LLM 编码常见反模式，是”对实习生的兜底”

讲师在视频末尾总结了一个核心观察：这些条目”有时候感觉像废话”，但”如果不加，AI 又好像真的不太懂”。系统提示词在这里扮演的角色，不是教一个资深工程师如何写代码，而是给一个**实习生**划定行为底线——告诉他什么能做、什么不能做、遇到困难时该怎么办。

从这个角度看，`getSimpleDoingTasksSection()` 的八条纪律与后续 `getActions` 中的风险三件套（blast radius / reversibility / approval scope）是同一种工程态度的不同抽象层：§1 在这里埋下”先理解再动手”的态度，§3 将把它具体落到”动作前的克制”上。下一节（§2）则从”不要过度抽象”这条规则出发，桥接到 `/help` 自我披露与 plugin 注入的产品哲学。

1.6 本章小结

本节从 `getSystemPrompt()` 的三目运算符入口出发，解析了 `getSimpleDoingTasksSection()` 生成的八条软件工程任务纪律：

1. **结合工作目录理解模糊指令**——模糊需求必须落地到实际代码，不能基于常识空回；
2. **先阅读再修改**——未读过的代码不许提建议；
3. **禁止凭空新建文件**——优先编辑既有文件，避免”一功能一脚本”的散乱架构；
4. **不要估时**——重点说明完成事项，而非给出不靠谱的时间预测；
5. **不盲目重试，求助优先于硬撑**——分析失败原因，陷入困境时主动向用户提问；
6. **安全优先，不越界优化**——不在用户要求外新增功能或重构；
7. **简洁代码，不乱加注释**——代码自解释，减少噪音与过时注释；
8. **不过度抽象、不留半成品**——三段相似代码才值得抽象，不为不可能场景写错误处理。

这些规则通过三目运算符按 `outputStyleConfig` 状态做条件挂载，既保证了编码风格下的行为一致性，又避免了与非编码 `output style` 发生冲突。它们看似琐碎，实则精准命中 LLM 编码的八大反模式，是系统提示词作为“实习生兜底手册”的典型体现。下一节 (§2) 将从“过早抽象”这条规则出发，展开讨论 `/help` 自我披露、飞书发布会类比，以及 `output style` 背后的 `plugin + hook` 动态注入机制。

2 过早抽象、自我披露与 Output Style 的 Plugin 哲学

2.1 问题：为什么系统提示词要管“代码该怎么写”

在 `prompt.ts` 的 `getSimpleDoingTasksSection` 中，有一段看似“给实习生的唠叨”：

- 先阅读再修改，不要对没读过的代码提建议；
- 不要凭空新建文件，优先编辑既有文件；
- 失败后别死磕，主动向用户提问优于继续硬试；
- 不为不可能的场景写错误处理、不留 TODO 半成品、不要乱加注释。

这些规则在 §1 已经以“条目级反模式”的形式出现。本节任务是：把其中一条——**过早抽象**——单独拎出来，看看它如何桥接到 Claude Code 更深层的产品哲学：**自我披露与 plugin 注入**。

过

早抽象 (premature abstraction) 是 LLM 编码最常见的反模式之一。系统提示词明确写道：**三段相似代码优于过早抽象**。复杂度应贴合任务本身，不要为假设未来的需求来做设计。

讲师在视频中解释，很多软件工程师一上来就喜欢搞大量接口，但代码是演化出来的，不是一开始就做过多过渡设计。对 LLM 而言尤其如此：模型没有“演化体感”，容易在看到两段重复代码时就急着提取抽象层，结果制造出不必要的间接性，反而让后续修改更困难。`prompt.ts` 把这条规则写进系统提示词，本质上是给模型一个**刹车信号**。

2.2 主张：好的智能体应该“在对话框里完成自我披露”

如果说“不要过早抽象”是面向代码的约束，那么接下来 `prompt.ts` 转向面向**用户交互**的约束。系统提示词明确告诉模型：

- 若用户需要反馈问题，可以告诉用户在这里来提交 issue；
- 若用户需要帮助，可以推荐 `/help` 这一 slash command；
- 若用户想进入 shell 直通模式，可以提示 `bash mode` (! 触发)。

这三件事的共同点是什么？答案是：**不逃离对话框**。用户不需要退出终端去翻文档、去 GitHub 搜 issue 模板、去官网查命令列表；模型在对话过程中就能完成自我能力的披露与引导。

好

的智能体应能“原地”告知用户自身能力：`/help`、issue 入口、`bash mode`、slash command。不要把用户踢出对话流。

讲师在这里插入了一个产品类比——2026 年飞书新品发布会上的”周星星”智能体演示。飞书机器人传统上需要到开放平台配置界面修改头像和名称，而发布会上演示的场景是：用户在对话框里直接说”以后你叫周星星”，智能体在对话中完成自我配置修改，头像和名称实时更新。这个案例与 Claude Code 的 /help 自我披露形成了产品哲学上的呼应：**能在这个里面进行反馈、配置和修改，就是不逃离对话框。**



图 4：飞书发布会演示：智能体在对话中完成自我配置修改，与 Claude Code 的 /help 自我披露形成产品哲学上的呼应。⁴

⁴ 视频画面时间区间：00:05:25-00:05:35。

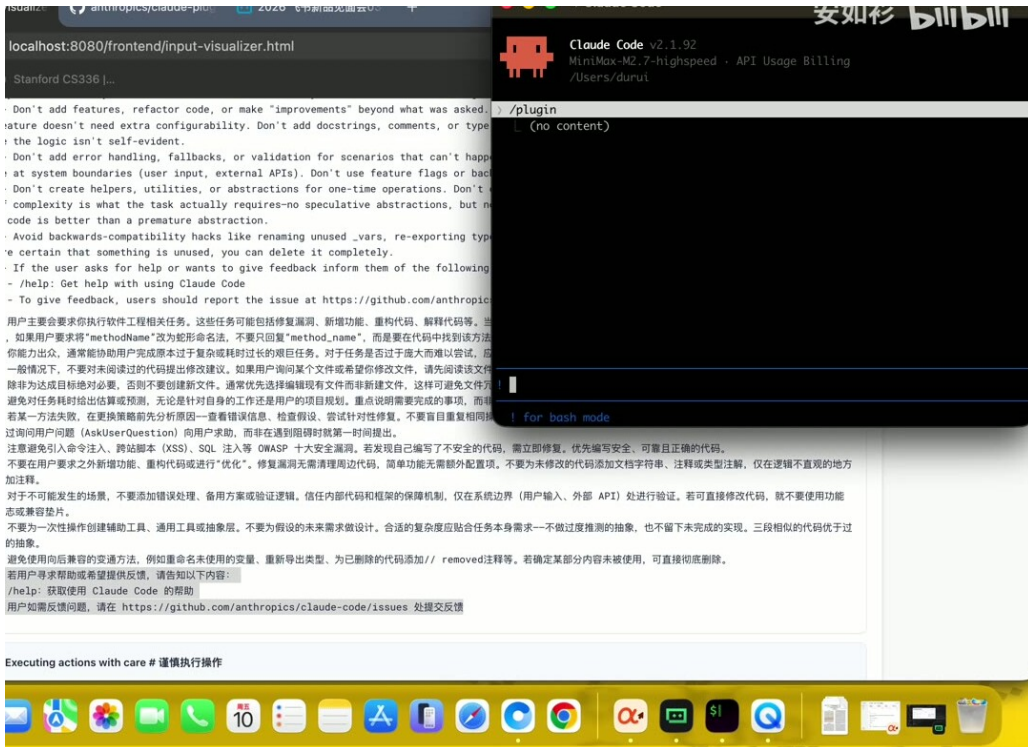


图 5: Claude Code 终端中的 `/plugin` 命令入口：插件系统把 skills 也当作产品来暴露给用户。⁵

2.3 机制：output style 不是硬编码，而是 plugin + hook 动态注入

讲到这里，视频抛出了一个关键问题：outputStyleConfigure 到底是从哪来的？

在 prompt.ts 外层，有一个 getter 叫 outputStyleConfigure。点击它的 get 函数，会发现它检查“有没有对应的 plugin 来做 output style 的设置”。这个 plugin 不在 Claude Code 主仓库里，而是在 Anthropic 官方插件仓库 anthropics/plugins-official 中，名字叫 learning-output-style。

O

output style 不是 prompt.ts 内置的硬编码，而是通过 plugin + hook 在每次对话开始前动态注入 additional context。plugin 系统与系统提示词是同一接口的两种填法。

具体机制是：plugin 在对话开启前以 hook 形式追加一段 instructions，从而改变 Claude 的行为基调。这与 §1 中三目运算符按 outputStyleConfig 状态做分支的逻辑是同一套架构的两面——静态分支在 prompt.ts 内部，动态注入由 plugin 在外层完成。

⁵ 视频画面时间区间：00:05:35–00:05:45。

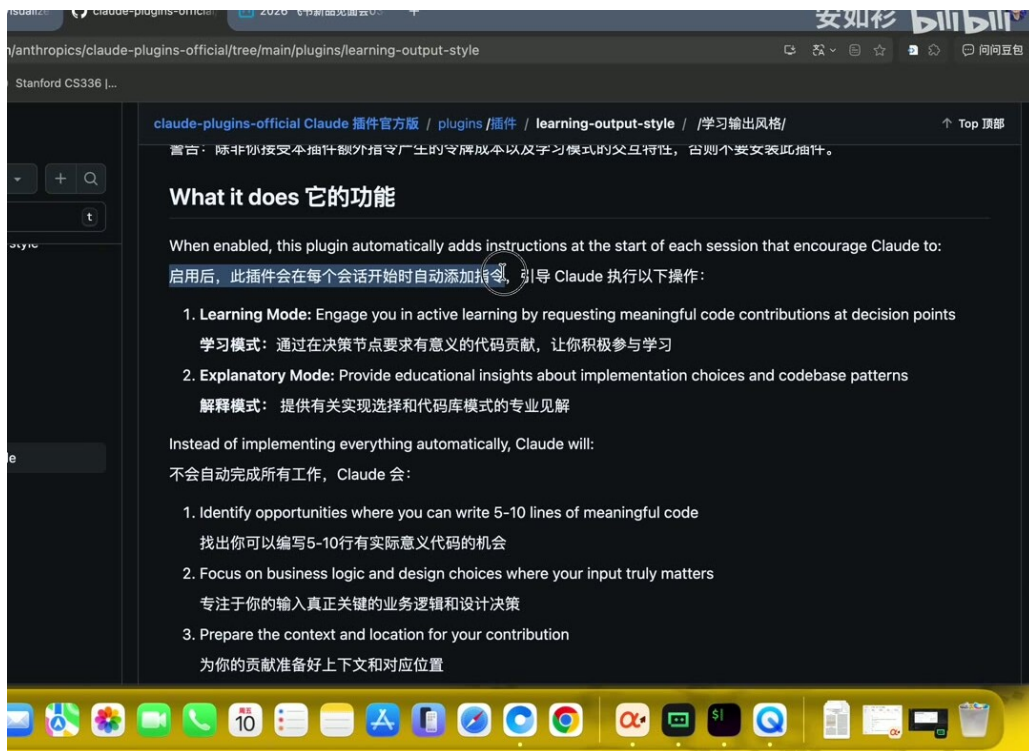


图 6: Anthropic 官方 learning-output-style 插件说明：两种模式（Learning / Explanatory）在会话开始时自动注入额外指令。⁶

2.4 示例：Learning output style 如何把 Claude 变成”小老师”

learning-output-style 插件的核心设计非常有趣：它把 Claude 从”代写者”改成”小老师”。当你遇到一个开发需求、或者在读一个开源项目时，Claude 不一定要帮你做完所有工作。它会找到一些你可以来写代码的部分——通常是 5-10 行有意义的代码——然后给你对应的上下文、指导你的实现、解释 trade-offs、再给出有启发性的见解。最后，**你自己来写代码**。

Learning output style 把 Claude 从”代写”改成”小老师”，让用户保留写代码的主动权。它的核心哲学是：*learning by doing is more effective than passive observation*。

这种设计与传统 AI 编程助手的”一键生成”逻辑形成了鲜明对比。传统模式下，用户被动观察模型生成的代码；而 Learning output style 强制模型保留”空白”给用户填，把学习效果放在了效率之前。

⁶ 视频画面时间区间：00:07:25–00:07:35。

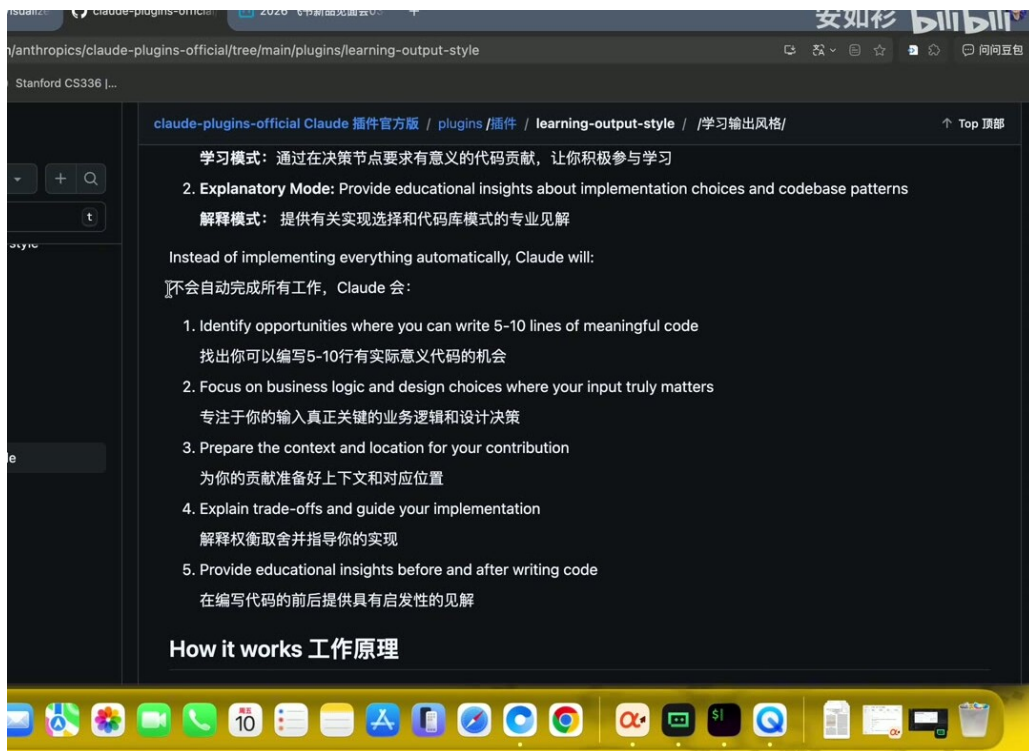


图 7: Learning output style 的核心哲学: Claude 不替你写完所有代码, 而是找出你可以贡献 5-10 行有意义代码的机会, 并给出指导。⁷

从源码视角看, `learning-output-style` 插件的 README 中列出了 5 条 Claude 行为约束:

1. Identify opportunities —— 找出用户可以动手写的代码机会;
2. Focus on business logic —— 聚焦业务逻辑而非样板代码;
3. Prepare context —— 为用户准备好必要的上下文;
4. Explain trade-offs —— 解释不同实现方案的取舍;
5. Provide insights —— 给出有启发性的见解。

这 5 条规则最终通过 plugin 的 hook 机制, 在每次对话开始前被追加到系统提示词中, 成为 `outputStyleConfigure` 的返回值来源。

2.5 收束: plugin 是被当成“产品”来做的

讲师在视频中特别强调了一个观察: Anthropic 把 plugins 和 skills 都当做产品来看待。它们不是简单的技术扩展点, 而是在看“用户到底需要什么”。

`learning-output-style` 的 README 写得像一份产品说明书, 有“What it does”、“How it works”、“Philosophy”三个板块——这种文档结构本身就说明, 官方希望用户把 plugin 当作可安装、可配置、可感知的产品功能, 而非隐藏在代码里的工程细节。

这与“飞书”“周星星”自修改的案例形成了闭环:

- 飞书让用户在对话框里配置智能体;

⁷ 视频画面时间区间: 00:07:35-00:07:55。

- Claude Code 让用户在对话框里通过 `/help`、`/plugin` 发现与配置能力；
- `learning-output-style` 让 Claude 在对话框里切换”代写模式”与”教学模式”。

三者共享同一个产品信念：**智能体的配置与反馈，应该内嵌于对话本身。**

2.6 本章小结

本节从 §1 的”三段相似代码优于过早抽象”出发，沿着两条线索展开：

1. **交互线索**：系统提示词把 `/help`、issue 入口、`bash mode`、`slash command` 写进模型行为规则，确保用户在卡住时不必逃离对话框——这与飞书”周星星”自修改的案例共享”原地配置”的产品哲学。
2. **架构线索**：`output style` 的源头不是 `prompt.ts` 的硬编码，而是 `outputStyleConfigure` getter 背后的 `plugin + hook` 机制；以 `learning-output-style` 为例，它在对话开启前动态注入 `additional context`，把 Claude 从”代写者”变成”小老师”。

最终落点是一个设计哲学：*learning by doing is more effective than passive observation*。plugin 不是装饰，而是被当成产品来做的能力扩展；它们与系统提示词共享同一套注入接口，只是填法不同——静态分支在 `prompt.ts` 内，动态追加在 `plugin` 中。下一节 (§3) 将沿着”动作前的克制”继续深入，讨论 `getActions` 如何把风险意识写进系统提示词。

3 Executing Actions with Care: `getActions` 的风险意识设计

3.1 问题：模型该怎么在执行动作前自己判断风险

在 `prompt.ts` 中，`getActionsSection` 紧跟 `getSimpleDoingTasksSection` 之后，负责装配一段名为 *Executing actions with care* 的系统提示词。它的核心问题非常直接：当模型拥有删文件、改代码、终止进程等能力时，它该如何在动手前自行判断——这件事我能不能做？需不需要先问用户？

讲师在视频中一针见血地指出：**如果你不告诉大模型什么叫高风险，他可能就直接 `rm -rf` 了。** LLM 没有人类的”后怕”经验，不会天然知道删除数据库和删除临时文件之间的区别。因此，`prompt.ts` 必须把风险判断的心智模型显式写进系统提示词，让模型在每次执行动作前都做一次自检。

3.2 主张：动作能否执行 = 可逆性 × 影响面 × 批准范围

`getActionsSection` 给出的判断框架可以概括为三个维度的综合评估：

- **可逆性 (Reversibility)**：这个动作能不能被撤销？编辑文件可以回滚，删除数据库很难恢复。
- **影响面 (Blast Radius)**：这个动作的潜在破坏半径有多大？改一行配置可能影响整个服务，改一个变量名只影响局部作用域。
- **批准范围 (Approval Scope)**：用户给过一次同意，是否意味着后续同类操作都可以自动执行？

这三个维度相乘，共同决定了一次动作是否需要显式征求用户同意。可逆性低、影响面大、批准范围模糊的操作，必须被拦截在”执行”之前。

风险判断三角：Blast Radius + Reversibility + Approval Scope

Blast radius (影响面) + *Reversibility* (可逆性) + *Approval Scope* (批准范围) 构成 `getActions` 的核心风险判断框架。三者不是独立 checklist，而是需要综合权衡的三角：一个影响面极小但完全不可逆的操作（如删除唯一密钥），仍然需要用户确认。

3.3 机制：危险操作必须主动征求同意，一次批准不传递

`getActionsSection` 的 prompt 原文中，明确列出了一系列”需要用户确认的高风险操作示例”。讲师在视频中逐条念出时，连用三个”血和泪”来形容这些规则背后的工程教训。

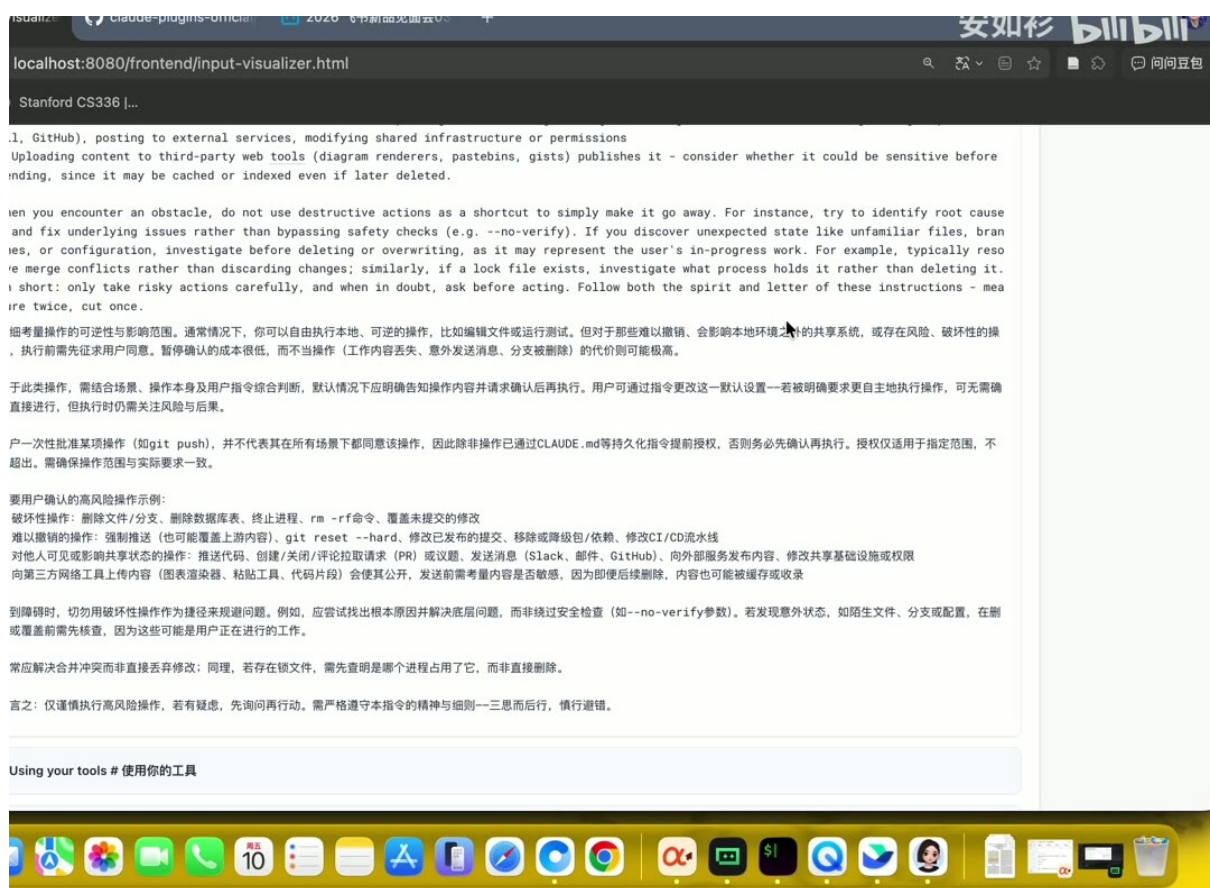


图 8: `getActionsSection` 中的高风险操作清单：系统提示词直接把”血和泪”的经验写成规则。⁸

这些高风险操作包括：删除文件、删除分支、删除数据库、终止进程、`rm -rf`、覆盖未提交的修改等。系统提示词不仅列出了清单，还给出了行为准则——暂停确认的成本很低，而不当的操作代价可能很高。

⁸ 视频画面时间区间：00:09:05–00:09:15。

高风险动作必须先确认再执行

删文件 / 删分支 / 删数据库 / 终止进程 / 覆盖未提交修改等操作, 属于 `getActionsSection` 明确标注的高风险动作清单。模型在执行前必须主动征求用户同意, 不能假设”用户让我改代码”就等于”用户允许我删数据库”。

更关键的是, 系统提示词还补了一条看似多余、实则重要的边界规则:

用户对一次操作的批准不构成对后续同类操作的批准

用户说”好, 删吧”, 只代表他同意这一次删除。下一回遇到同类场景, 模型必须重新确认。这是防止”批准滑坡”的保险栓——否则模型可能在用户一次点头后, 连续执行一系列破坏性操作。

3.4 示例: 实习生类比——若不告知, 他就直接 `rm -rf` 了

讲师在视频中用了一个非常生动的类比来解释为什么必须显式列出高风险操作:

”接下来假如说你跟实习生说, 这些实习生也不懂啊, 什么叫高风险的操作啊? 哎那个时候就需要再给他个例子说: 比如说删除文件、删除分支、删除数据库……哎呀这都是血和泪。”

这个类比精准地揭示了系统提示词的设计动机。实习生(模型)有动手能力, 但缺乏经验判断力; 如果你不给他一份”什么不能做”的清单, 他就会凭直觉行事。而 LLM 的”直觉”来自预训练数据中的统计模式, 并不包含你公司生产环境的具体风险边界。

`getActionsSection` 中的高风险清单, 本质上是一份**写给实习生的红线条例**。它不是抽象的原则, 而是具体到”删除数据库””覆盖未提交修改”的场景级指令——因为模型对抽象原则的理解往往不稳定, 对具体示例的遵循率更高。

3.5 收束: spirit and letter / measure twice, cut once

`getActionsSection` 的末尾有两句话, 被讲师特意从”英语文学的角度”拎出来品味:

Follow both the spirit and the letter of these instructions. Measure twice, cut once.

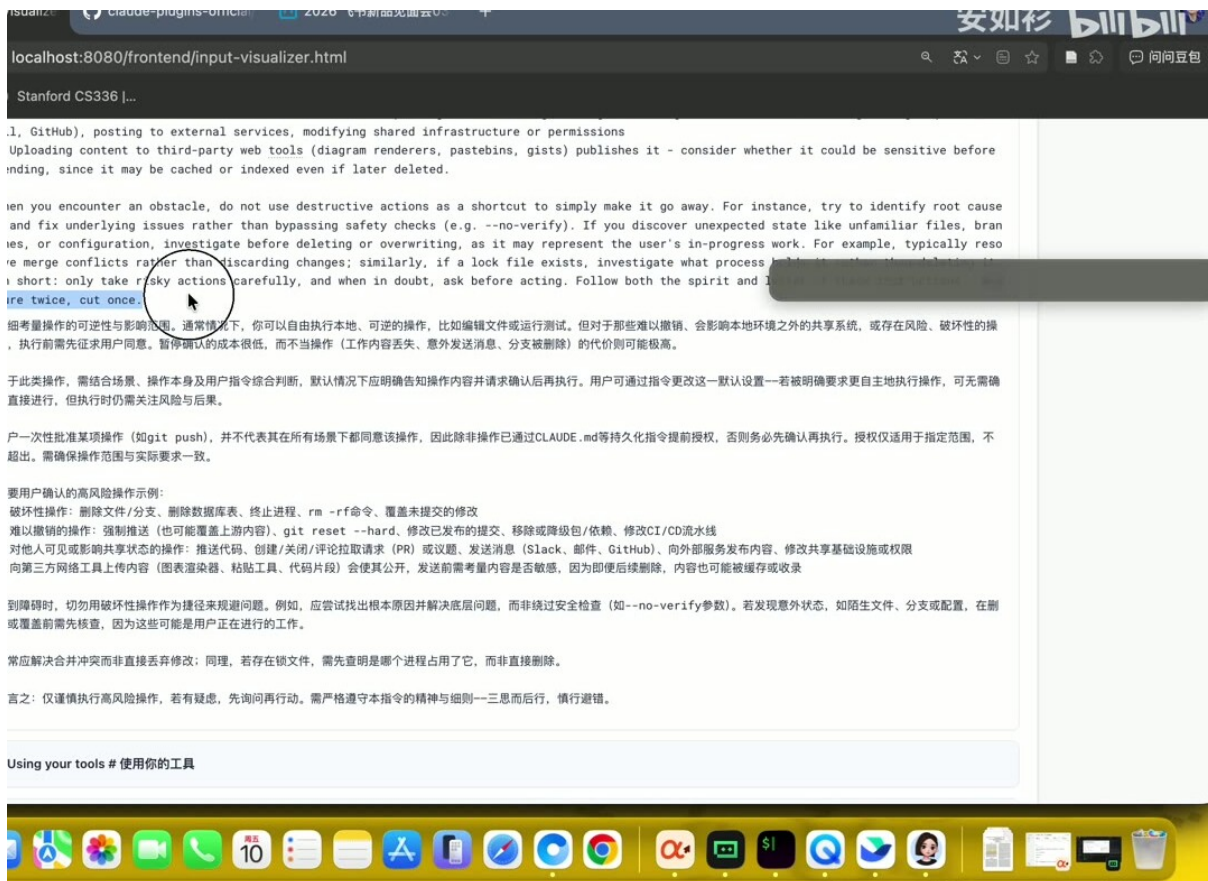


图 9：系统提示词的文学性结尾：不仅要看字面意思 (letter)，更要体会背后的谨慎精神 (spirit) ——三思而后行。⁹

Spirit and letter 的深层含义是：模型不能死抠字面——“你只说了删数据库要确认，那删表呢？”——而要体会背后的**谨慎精神**。凡是与所列示例在风险特征上相似的操作，都应被同等对待。这是一种要求模型进行**类推** (analogy) 而非**枚举匹配** (enumeration) 的提示词文学。

Measure twice, cut once 则是木匠谚语“量两次锯一次”的直译，对应中文的“三思而后行”。它把工程中的谨慎态度压缩成一句朗朗上口的指令，让模型在每次动手前都多停一轮、多想一层。

Spirit and letter / measure twice, cut once —— 提示词的“文学性”

Spirit and letter 与 *measure twice, cut once* 的提示词“文学性”在于：它们不是让模型死抠示例，而是逼模型**类推**背后的谨慎精神。好的系统提示词不仅要列规则，还要让模型学会“举一反三”。

3.6 风险矩阵：可逆性 vs. 影响面的四象限分析

为了更直观地理解 `getActions` 的风险判断逻辑，我们可以把“可逆性”和“影响面”画成一张 2×2 风险矩阵。横轴为影响面 (Blast Radius)，从左到右由小变大；纵轴为可逆性 (Reversibility)，从下到上由难到易。

⁹ 视频画面时间区间：00:10:25–00:10:35。

1. **判断框架**：以 *blast radius*（影响面） $\times$ *reversibility*（可逆性） $\times$ *approval scope*（批准范围）三件套作为动作能否执行的决策依据。
2. **红线条例**：把删除文件、删除分支、删除数据库、终止进程、覆盖未提交修改等”血和泪”的经验写成显式清单，防止模型凭直觉越界。
3. **批准边界**：明确一次同意不传递给后续操作，堵住”批准滑坡”的漏洞。
4. **文学收束**：以 *spirit and letter* 与 *measure twice, cut once* 收尾，要求模型既守字面、更会类推，把谨慎精神内化为行为基调。

这与 §1 已埋下的”先理解再动手”的态度一脉相承——§1 讲的是”读代码的克制”，§3 讲的是”执行动作的克制”，是同一种工程态度在不同抽象层的落地。下一节（§4）将进一步把”可逆性”从抽象原则落到工程实现，看看 Edit 工具如何通过记录行号与差异，让 conversation 可以 *rewind* 到上一轮状态。

4 getTools：专用工具为何优于原始 Bash 命令

4.1 问题：既然 Bash 已经够用，为什么还要 Read / Edit / Grep

在 `prompt.ts` 中，`getUsingYourToolsSection` 紧跟 `getActionsSection` 之后。源码层面，这两个函数定义相邻，体现”先讲原则、再讲方法”的结构设计。

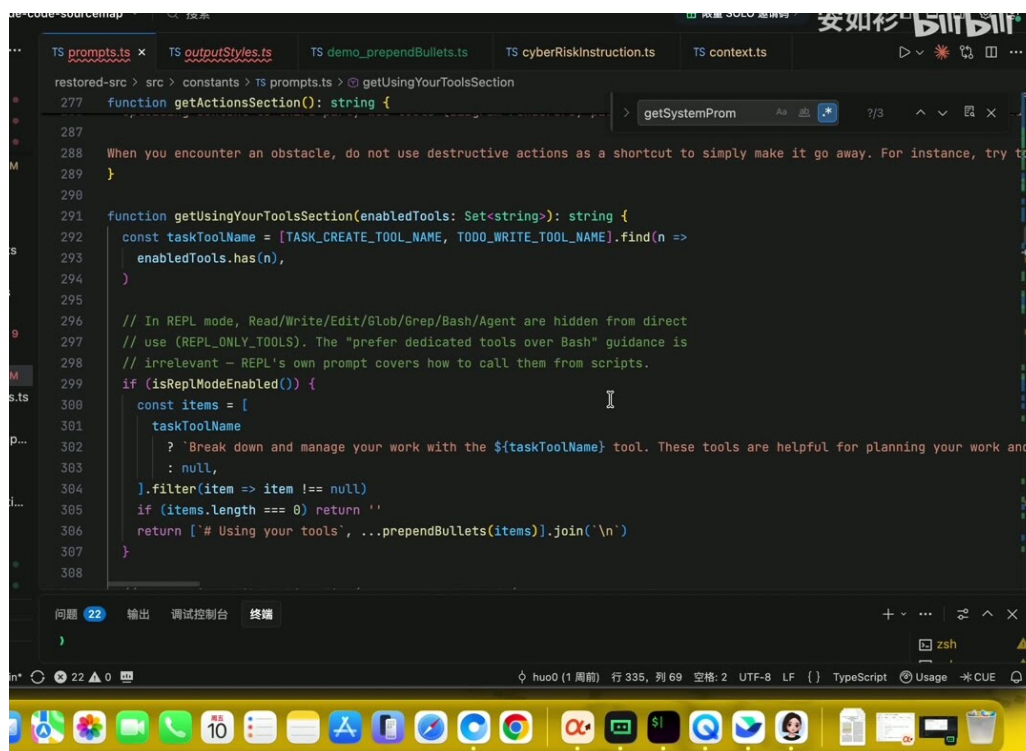


图 11: 源码视角：`getActionsSection` 与 `getUsingYourToolsSection` 在 `prompt.ts` 中紧密相邻，体现”先讲原则、再讲方法”的结构设计。¹¹

¹¹ 视频画面时间区间：00:11:05–00:11:15。

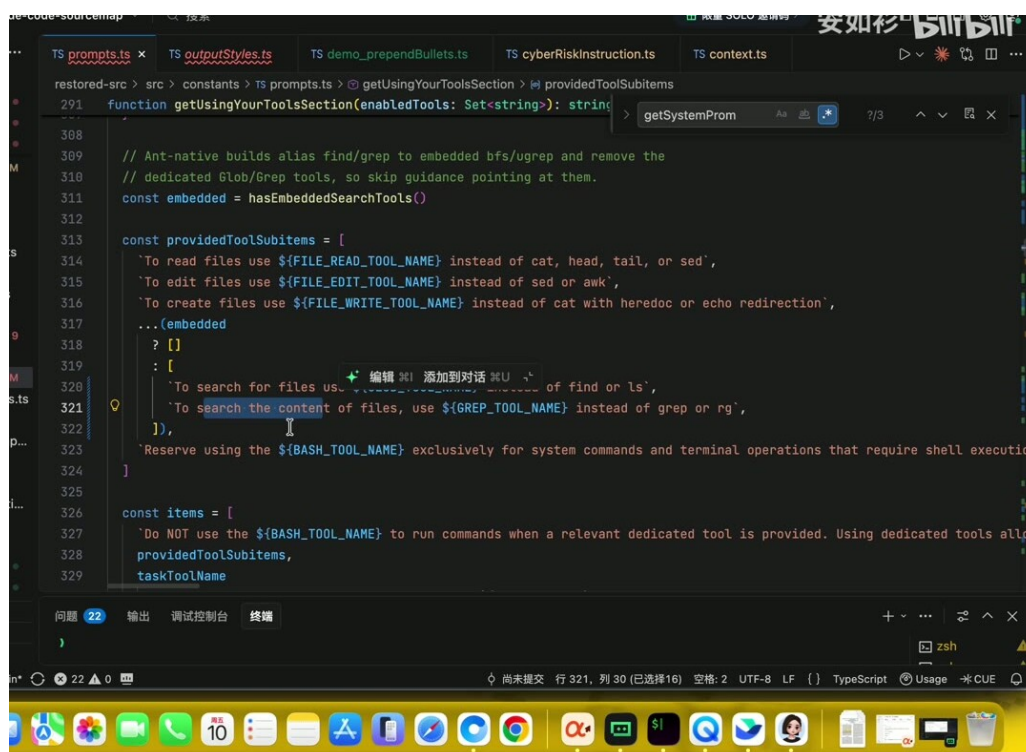
进入 `getUsingYourToolsSection` 后，系统提示词首先列出默认挂载的工具集合。讲师在视频中将其逐一念出：

- Read —— 读取文件内容；
- Edit —— 编辑既有文件；
- Create —— 创建新文件；
- Search —— 按文件名搜索；
- Grep —— 按内容搜索；
- Agent —— 调用子 agent 完成子任务。

这些工具并非可选项，而是 Claude Code 默认就会注入系统提示词的 `enabled tools`。接下来的问题是：既然 Bash 工具已经能让模型执行任意 shell 命令，为什么还要在 prompt 里额外挂载这一套专用工具？直接用 `cat`、`sed`、`awk`、`find`、`grep` 不就可以了吗？

4.2 主张：专用工具不是 Bash 的”语法糖”

讲师给出的答案是：专用工具绝非原始 Bash 命令的简单包装。系统提示词明确要求模型优先使用 Read 代替 `cat/head/tail/sed`，优先使用 Grep 代替 `grep`，优先使用 Edit 代替 `sed/awk` 编辑文件。



```
291 function getUsingYourToolsSection(enabledTools: Set<string>): string {
308
309 // Ant-native builds alias find/grep to embedded bfs/ugrep and remove the
310 // dedicated Glob/Grep tools, so skip guidance pointing at them.
311 const embedded = hasEmbeddedSearchTools()
312
313 const providedToolSubitems = [
314   'To read files use ${FILE_READ_TOOL_NAME} instead of cat, head, tail, or sed',
315   'To edit files use ${FILE_EDIT_TOOL_NAME} instead of sed or awk',
316   'To create files use ${FILE_WRITE_TOOL_NAME} instead of cat with heredoc or echo redirection',
317   ...(embedded
318     ? []
319     : [
320       'To search for files use ${BASH_TOOL_NAME} instead of find or ls',
321       'To search the content of files, use ${GREP_TOOL_NAME} instead of grep or rg',
322     ]),
323   'Reserve using the ${BASH_TOOL_NAME} exclusively for system commands and terminal operations that require shell execution'
324 ]
325
326 const items = [
327   'Do NOT use the ${BASH_TOOL_NAME} to run commands when a relevant dedicated tool is provided. Using dedicated tools allow
328   providedToolSubitems,
329   taskToolName
330 ]
331 }
```

图 12: `getUsingYourToolsSection` 源码：系统提示词明确要求使用专用工具（Read/Edit/Grep）而非原始 Bash 命令。¹²

¹²视频画面时间区间：00:11:30–00:11:40。

这一主张可以从四个维度展开论证：用户理解审阅、可逆回滚与审计、模型准确率与 harness 可调校性、跨平台一致性。下面逐条说明。

专用工具 vs 裸 Bash 的四大动机

专用工具相对原始 Bash 命令在以下四件事上更优：**理解审阅、回滚、审计、跨平台一致性。**

4.2.1 维度一：用户理解审阅

第一个动机是**让用户看得懂**。讲师在视频中举了一个非常实际的例子：大部分计算机专业的学生甚至都没听过 `sed` 和 `awk`，但他们都看得懂什么叫 `Edit`。

专用工具的命名本身就是用户体验设计。当模型调用 `Edit` 时，前端 UI 可以把这个动作渲染成一个友好的 diff 视图——增删改行高亮显示、行号对齐、变更范围一目了然。如果模型用的是 `sed -i 's/foo/bar/g'`，用户要么看不懂，要么需要额外的心理翻译成本。

工具命名也是用户体验

用户大多看不懂 `sed/awk`，但都懂 `Edit`。专用工具的命名与参数设计，本质上是把“机器友好的 Unix 命令”翻译成“人类可审阅的操作语义”。

4.2.2 维度二：可逆回滚与审计

第二个动机是**可逆性**。讲师指出，专用工具记录了编辑的行数与编辑的内容，这让系统拥有了更好的回滚能力。

具体来说，当 `Edit` 工具执行一次修改时，它会精确记录：

- 目标文件路径；
- 修改的起止行号；
- 旧文本与新文本的差异。

这些信息被结构化地保存下来后，系统可以 `rewind` 到上一轮 conversation——只要把 `Edit` 队列中对应的条目删掉，文件状态即可恢复。这种回滚不是基于文件系统快照的笨重操作，而是基于**结构化编辑日志**的轻量撤销。

同时，结构化日志也让审计成为可能。管理员或用户可以随时查看“模型在这一轮对话中到底改了哪些文件的哪些行”，而不是从一堆 `sed` 输出中反推变更范围。

4.2.3 维度三：模型准确率与 harness 可调校性

第三个动机是 **Harness 本身可以成为优化对象**。讲师在这里引用了一个著名案例：

“I improved 15 LLMs at Coding in one afternoon. Only the harness changed.”

这句话来自 Can Boluk 的博客。核心发现是：同一组开源模型，仅替换 `read/edit` 工具的 harness（即工具的实现格式与调用协议），编码 benchmark 的准确率就显著提升。

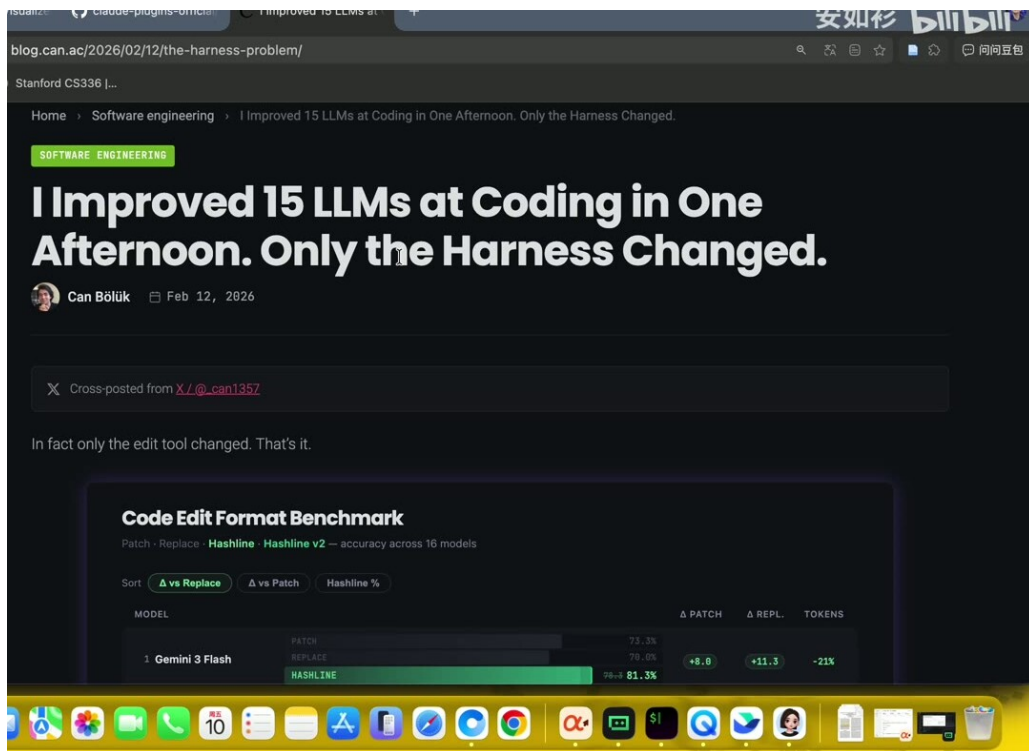


图 13: Harness 效应的经典案例：仅改变编辑工具的格式（edit harness），15 个开源模型的编码准确率同时提升。¹³

这个案例的教学含义非常深刻：

1. **Harness 与模型权重正交**。不需要重新训练模型，也不需要更换更大的模型，仅优化外围工具脚手架就能涨分。
2. **工具设计是 agent 性能的杠杆**。同样的模型，用不同的 edit format（Patch vs. Replace vs. Hashline），结果可能天差地别。
3. **专用工具预留了迭代接口**。prompt.ts 向模型暴露的是 Read/Edit 的协议层，底层实现可以继续实验、持续优化，而不影响系统提示词的文本。

¹³视频画面时间区间：00:13:35–00:13:45。



图 14: Hashline 编辑格式 benchmark: 在 Gemini 3 Flash、Claude Haiku 4.5、Kimi K2.5 等模型上, 专用编辑格式带来的准确率提升。¹⁴

Harness 一词的工程含义

Anthropic 把”模型外围的工具脚手架”(工具实现 + 调度 + I/O 格式) 作为独立优化对象, 与模型权重正交。Harness 可独立迭代, 是 agent 性能除模型本身之外的第二杠杆。

4.2.4 维度四: 跨平台一致性

第四个动机是**跨平台一致性**。讲师在视频中明确指出: Windows、Linux、macOS 上的命令行为并不相同。

例如:

- `find` 的语法在 BSD (macOS) 和 GNU (Linux) 之间有显著差异;
- `sed -i` 在 macOS 上需要额外指定备份后缀;
- `grep` 的递归选项、颜色输出、行号格式在不同系统上表现不一;
- Windows 上可能根本没有这些命令, 或需要 WSL/Cygwin 兼容层。

如果模型依赖裸 Bash 命令, 系统提示词就必须在每次对话时额外告知”你当前在什么系统上、该用哪种语法”。这种现场教学不仅浪费 token, 还容易出错。相比之下, 专用工具把平台差异封装在 harness 内部: 模型永远只调用 `Grep`, 至于底层用的是 `grep`、`ugrep` 还是 `bfs`, 由 harness 根据当前平台自行决定。

¹⁴ 视频画面时间区间: 00:13:45-00:14:00。

跨平台差异不可忽视

Bash 命令在 Windows / Linux / macOS 上行为不一致，靠 prompt 现场告知不如直接封装专用工具。平台差异应该被隔离在 harness 实现层，而不是泄露给系统提示词。

4.3 机制：prompt.ts 把工具集合作为”可替换的接口”

综合以上四个维度，专用工具的设计动机已经清晰。但还有一个架构层面的问题：为什么 prompt.ts 要”列出”这些工具，而不是直接让模型用 Bash 自己发挥？

答案是：**prompt.ts 把工具集合当作一种”可替换的接口”暴露给上层。**

在源码层面，getUsingYourToolsSection 中的 providedToolSubitems 数组就是这一接口的具体形态。它告诉模型”你现在有哪些能力可用”，但并不要求模型关心这些能力底层是如何实现的。内部可以用 bfs 替代 find、用 ugrep 替代 grep、用 Hashline 格式替代 Patch 格式——只要协议层不变，系统提示词就不需要改动。

Dedicated Tools vs. Raw Bash Commands		
Why Claude Code provides Read/Edit/Grep instead of cat/sed/grep		
	Dedicated Tool (e.g., Edit)	Raw Bash (e.g., sed/awk)
User Comprehension	High: UI shows diff clearly	Low: requires Unix expertise
Audit / Rollback	Structured: line-level edits recorded	Unstructured: hard to rewind
Cross-Platform	Consistent: same API everywhere	Fragmented: cmd differs by OS
Model Accuracy	High: constrained JSON schema	Variable: free-form shell output
Harness Effect	Tunable: format changes boost scores	Fixed: limited by shell syntax

Conclusion: Dedicated tools are not "wrappers" -- they are a deliberately designed interface layer.

图 15: 专用工具不是简单的”Bash 包装”——它在用户理解、审计回滚、跨平台一致性和模型准确率上均优于原始命令。¹⁵

上图从五个维度对比了专用工具与原始 Bash 命令的差异：

1. **用户理解**：专用工具以人类可读的语义命名 (Edit)，Bash 命令以机器友好的缩写为主 (sed)；
2. **审计回滚**：专用工具记录结构化行号差异，支持 rewind；Bash 命令的变更历史散落在 shell 输出中，难以追溯；
3. **跨平台一致性**：专用工具由 harness 屏蔽平台差异；Bash 命令需要模型自行处理 BSD/GNU/Windows 差异；
4. **模型准确率**：专用工具的 harness 可独立优化（如 Hashline 格式），Bash 命令的格式受限于 shell 协议，难以微调；
5. **Harness 可调校性**：专用工具的协议层与实现层解耦，可 A/B 测试不同格式；Bash 命令的 I/O 格式固定，实验空间小。

¹⁵ 视频画面时间区间：00:11:30–00:11:40。

4.4 本章小结

本节从 `getUsingYourToolsSection` 的默认工具列表出发，回答了”为什么不用纯 Bash”这一核心问题。

1. **用户理解**：专用工具的命名与结构化参数让用户可审阅、可理解，降低了 `sed/awk` 等命令的认知门槛。
2. **可逆回滚与审计**：`Edit` 工具精确记录行号与差异，支持 `rewind` 到上一轮 conversation，也为审计提供结构化日志。
3. **Harness 可调校性**：仅替换 `read/edit harness` 就能在 15 个开源模型上显著涨分，说明工具设计本身是 agent 性能的杠杆；`harness` 与模型权重正交，可独立迭代。
4. **跨平台一致性**：专用工具把 Windows / Linux / macOS 的差异封装在 `harness` 内部，避免让系统提示词承担”现场教学平台差异”的负担。

最终，`prompt.ts` 把 `Read/Edit/Create/Search/Grep/Agent` 这一组工具作为**可替换的接口**暴露给模型，而把具体实现留在 `harness` 层持续演化。这与 §3 中”可逆性”的抽象原则一脉相承——§3 讲”动作前要判断能否撤销”，§4 则以 `Edit` 工具记录行号、可 `rewind` 上一轮 conversation 给出了工程落地。下一节 (§5) 将进一步把这些原子工具聚合成单一的 REPL 工具，把原子调用降级为虚拟消息，实现上下文层面的进一步抽象。

5 TodoWrite、REPL 聚合、并行调用与 Tone & Style 收尾

5.1 任务列表与并行工具调用

在 `'prompt.ts'` 的 `'getUsingYourToolsSection'` 末尾，除了 `'Read'`、`'Edit'`、`'Grep'`、`'Bash'` 等原子工具之外，还有一个名为 `'TodoWrite'` 的工具。它的作用是把多步任务显式登记为待办列表，并支持逐项打勾（标记为完成）。讲师将其通俗地比作”最早期的 to-do list”——模型可以在一次会话中创建任务、标记完成，让用户对当前进度一目了然。

与 `'TodoWrite'` 同等重要的，是系统提示词中关于**并行工具调用**（parallel tool use）的说明。模型在单次回复中，可以同时发起多个工具调用，但前提是这些调用之间没有依赖关系。例如，若需要读取两个互不相关的文件，模型可以一次性发出两个 `'Read'` 调用，由 `harness` 并发执行，从而缩短整体交互轮数。

并行调用的前提：无依赖关系

单轮可并行调用多工具，**前提是它们没有依赖关系**。若后一步需要前一步的返回结果，则必须串行等待，不可强行并行。

这种设计体现了 Claude Code 对”效率”与”正确性”的权衡：并行能节省时间，但依赖关系必须由模型自行判断，不能为了快而乱。`'TodoWrite'` 的存在则让模型在并行执行之余，仍能保持对整体任务结构的显式追踪。

5.2 REPL 聚合工具：把原子调用降级为虚拟消息

‘getUsingYourToolsSection’ 中还有一个值得深挖的条件分支：‘isReplModeEnabled()’。当 REPL 模式开启时，系统提示词的行为会发生显著变化——‘Read’、‘Write’、‘Edit’、‘Glob’、‘Grep’、‘Bash’、‘Agent’ 等原子工具被**隐藏**，不再直接暴露给模型使用。

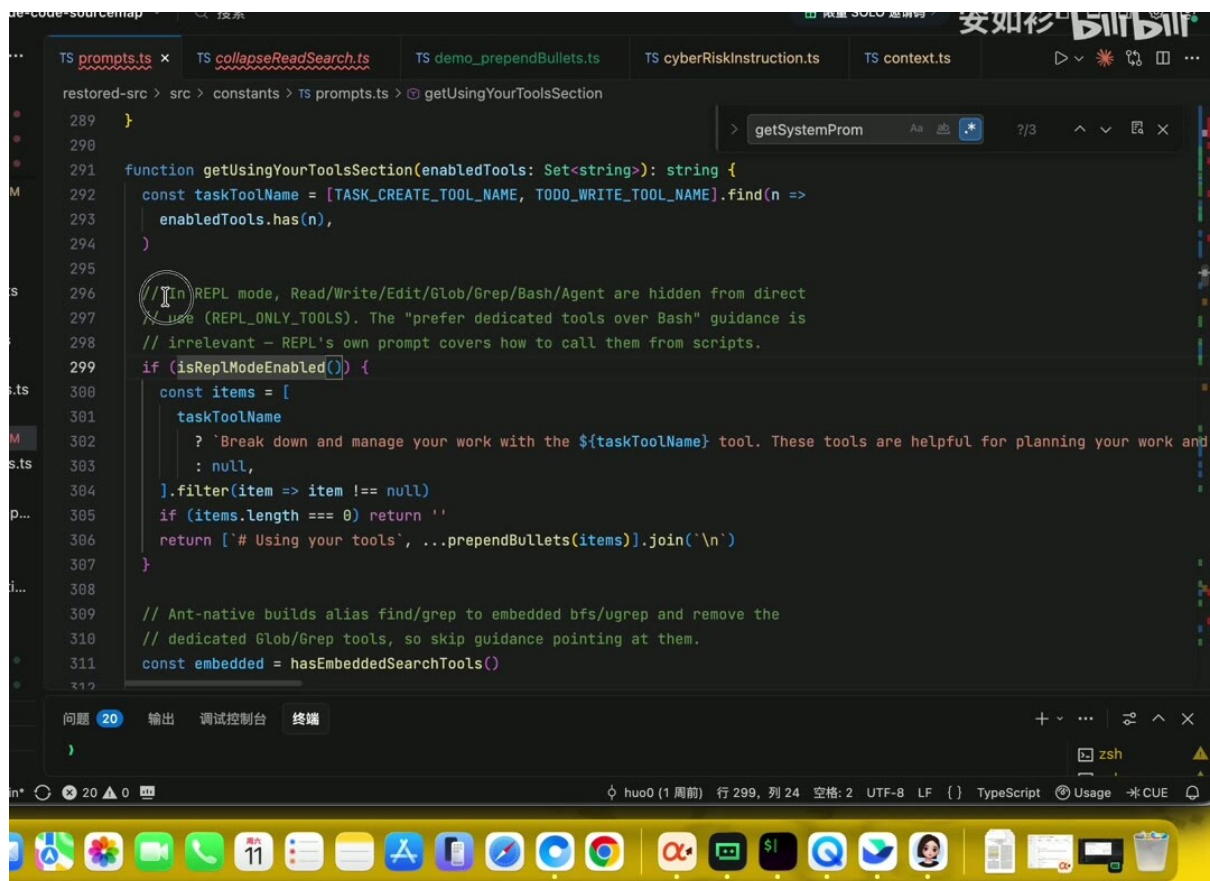


图 16: REPL 模式的源码注释：原子工具被隐藏，只能通过 REPL 脚本间接调用。¹⁶

为什么会这样？因为 REPL 模式下，这些原子工具被**聚合**进了一个外层工具——REPL 工具。模型不再直接调用 ‘Read’ 或 ‘Edit’，而是向 REPL 工具发送一段脚本，由 REPL 工具内部调度原子工具完成实际工作。

¹⁶ 视频画面时间区间：00:15:45–00:15:55。

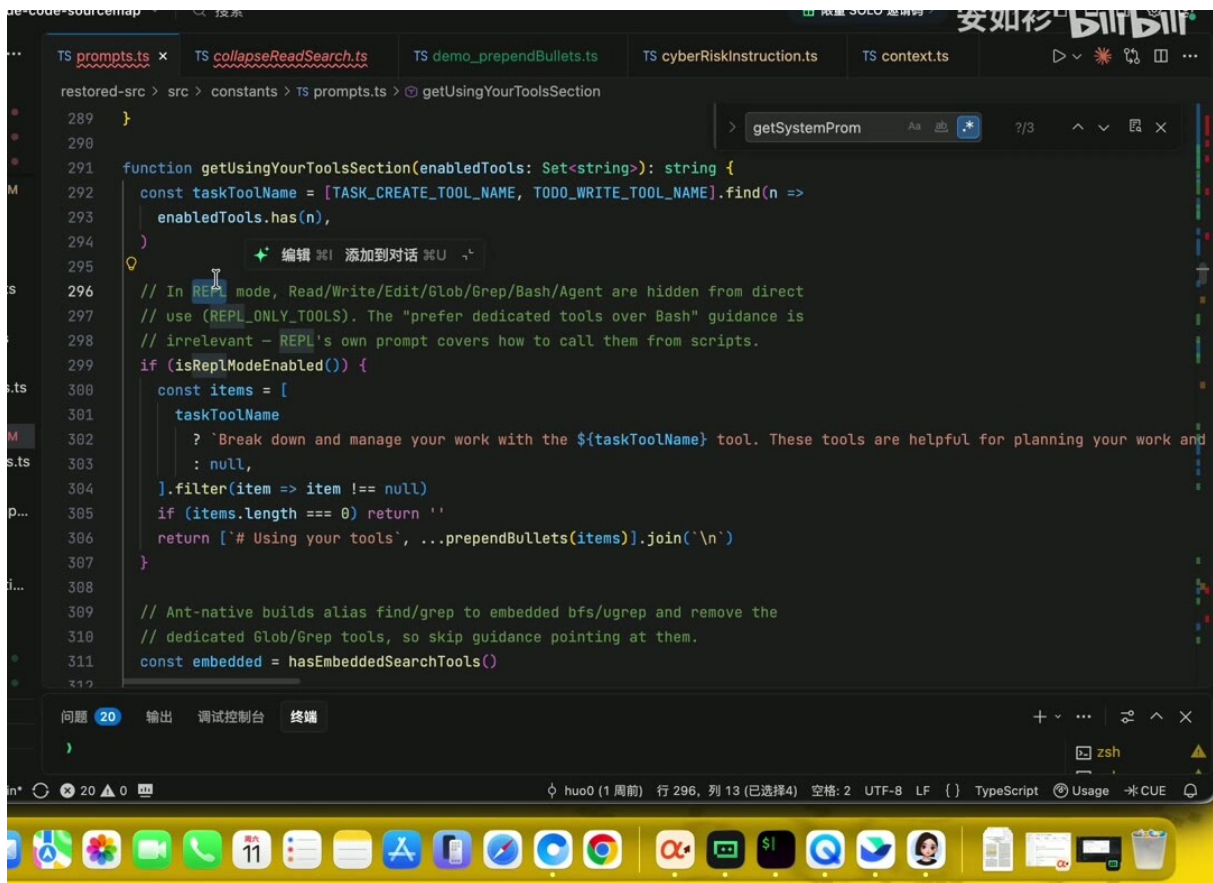


图 17: ‘isReplModeEnabled()’ 条件分支: 当 REPL 开启时, 系统提示词只保留任务管理工具, 原子工具全部收归 REPL 内部调度。¹⁷

这种聚合的核心价值在于上下文**减负**。在普通模式下, 每一次原子工具调用都会以真实的 tool result 形式回到 LLM 上下文, 占据 token 预算; 而在 REPL 模式下, 原子工具调用被建模为**虚拟消息** (virtual message) ——它们不是真实的 user/assistant turn, 但仍参与上下文记账。这样, harness 可以把多次原子调用的中间结果在内部消化, 只把最终聚合结果回传给 LLM, 从而显著压缩上下文长度。

REPL 聚合工具的设计本质

REPL 聚合工具把”多个原子工具调用”建模为”一个 REPL 工具的多次虚拟消息”, 是工具协议的进一步抽象。它把上下文负担从 LLM 侧转移到 harness 侧, 让模型专注于高层意图, 而非逐条工具调用的细节。

¹⁷ 视频画面时间区间: 00:15:55–00:16:05。

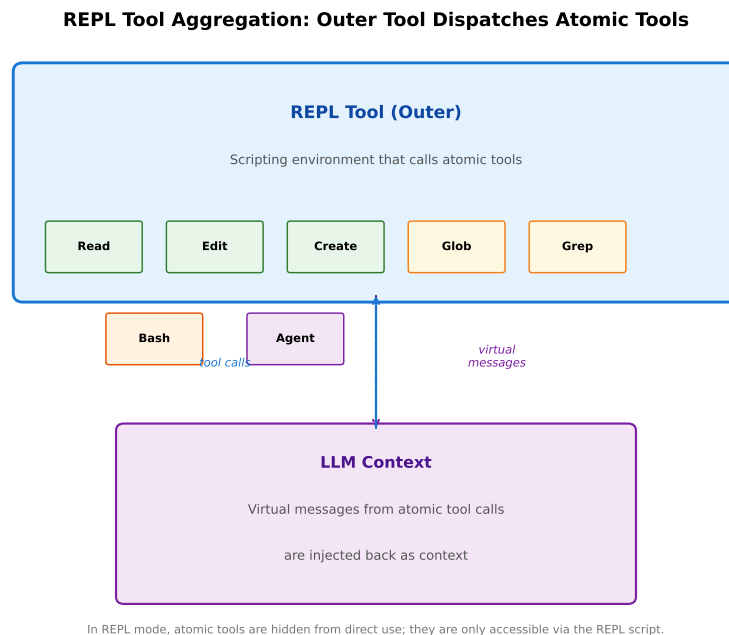


图 18: REPL 聚合架构：外层 REPL 脚本调用原子工具，工具返回的虚拟消息重新注入 LLM 上下文，形成闭环。¹⁸

上图展示了这一信息流的完整闭环：LLM 生成 REPL 脚本 → REPL 工具内部调度 ‘Read’/‘Edit’/‘Grep’ 等原子工具 → 原子工具返回结果以 virtual message 形式重新注入 LLM 上下文 → LLM 基于更新后的上下文继续生成。这种设计让 REPL 模式兼具”交互灵活性”与”上下文紧凑性”，是 §4 所列原子工具列表的进一步抽象。

5.3 工具接口的持续优化：bfs 与 ugrep

在讲解完 REPL 聚合后，讲师顺带提到了 Anthropic 内部的一个工程细节：他们会用 ‘bfs’ 替代 ‘find’，用 ‘ugrep’ 替代 ‘grep’，作为底层文件搜索与内容搜索的实现。

Anthropic 内部的工具迭代

Anthropic 内部以 ‘bfs’（替代 ‘find’）与 ‘ugrep’（替代 ‘grep’）作为底层搜索工具，性能与一致性更好。这说明工具接口是可被持续优化的——prompt.ts 中向模型暴露的只是 ‘Search’ 与 ‘Grep’ 的协议层，底层实现可以自由替换，而不影响系统提示词的文本。

这一点与 §4 中”harness 与模型权重正交”的论述形成呼应：工具协议是接口，实现是细节。只要接口不变，内部可以用更快的算法、更稳定的库、甚至完全不同的架构来替换，这正是把工具设计为**专用抽象层**而非裸 Bash 命令的长远收益。

5.4 Tone & Style 与输出风范

‘prompt.ts’ 的最后两个 section——‘getEnvInfo’ 与 tone-and-style——相对简单，因为它们返回的都是纯字符串，没有复杂的条件分支。但”简单”不代表”不重要”。讲师特意把镜头切到 tone-and-style 的文本，逐条讲解 Claude Code 对模型**输出风格**的硬性约束。

¹⁸视频画面时间区间：00:15:55–00:16:05。

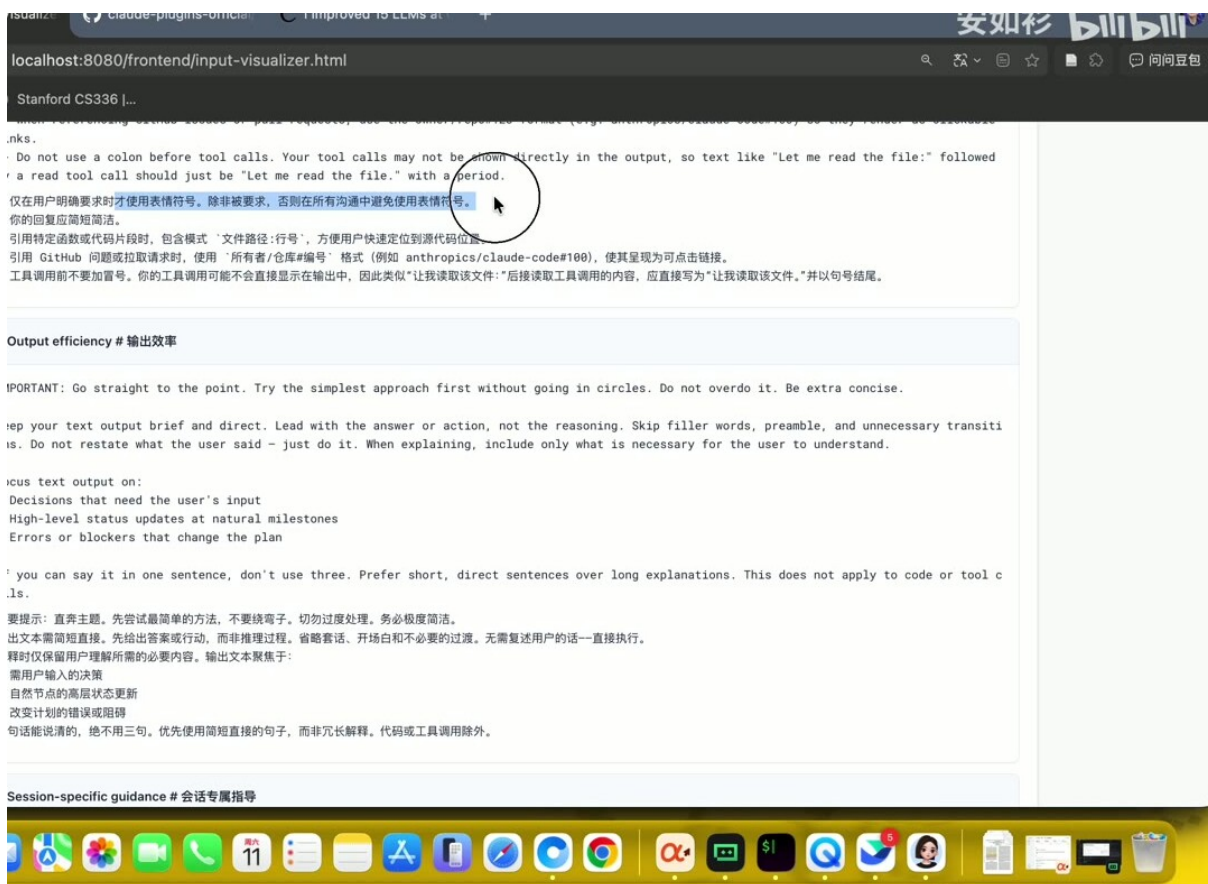


图 19: ‘getOutputEfficiencySection’: 输出效率规则——直奔主题、省略套话、用简短直接的句子而非冗长解释。¹⁹

这些规则可以归纳为五条核心输出风范：

1. **直奔主题**：不要绕弯子，避免开场白与套话；
2. **文件引用用 markdown 链接**：如 ‘[filename](path/to/file)’，方便用户直接点击跳转；
3. **GitHub issue 使用专属格式**：如 ‘owner/repo#123’，而非冗长的 URL；
4. **工具调用前不加冒号**：因为工具调用结果不会直接显示给用户，冒号会留下悬空表述；
5. **简短直接**：用句号结尾的短句，而非冗长解释。

¹⁹ 视频画面时间区间：00:17:00–00:17:15。

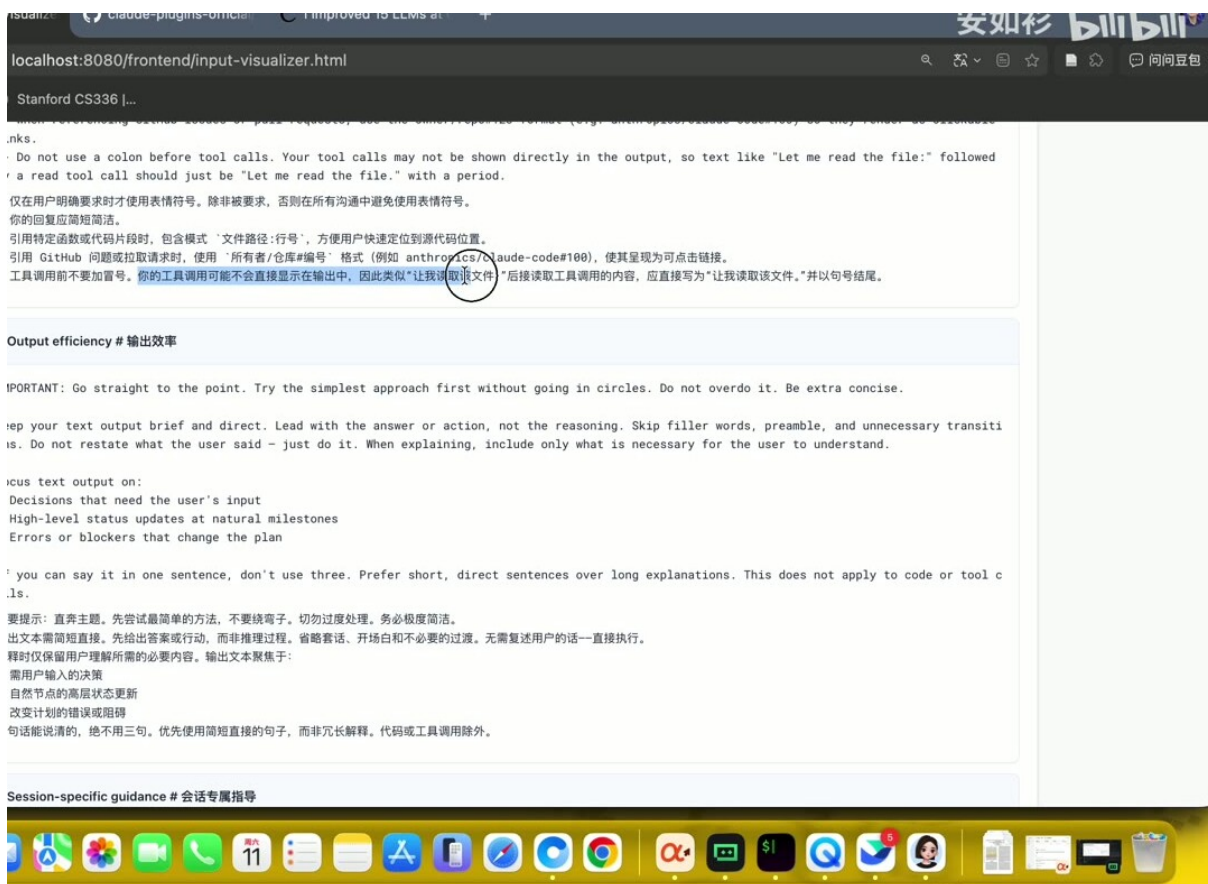


图 20: Tone and style 的细节规则：工具调用前不加冒号、文件引用用 markdown 链接格式、GitHub issue 用 ‘owner/repo#123’。²⁰

工具调用前以句号结尾而非冒号

工具调用前以**句号**结尾而非冒号；工具调用结果不会直接显示给用户，避免悬空表述。例如应写”让我读取该文件。”而非”让我读取该文件：”。

输出风格是用户感知的根因

tone-and-style 中的”直奔主题、避免开场白与套话”对模型生成有强约束作用，是用户感知”Claude Code 风格简洁”的根因。这些看似琐碎的格式规则，实际上构成了产品人格的底线。

讲师在视频末尾特别强调了”代码或工具调用除外”——即上述简洁规则不适用于代码块和工具调用本身，因为代码需要完整、工具调用需要结构化，不能为了简洁而牺牲正确性。

5.5 本章小结

本章从 ‘TodoWrite’ 与并行工具调用切入，揭示了 Claude Code 如何在单轮回复中高效调度多工具；随后深入 REPL 聚合工具的设计，阐释了”把原子工具调用降级为虚拟消息”这一抽象如何把上下文负担从 LLM 侧转移到 harness 侧；接着以 Anthropic 内部用 ‘bfs’/‘ugrep’ 替代 ‘find’/‘grep’

²⁰视频画面时间区间：00:17:15–00:17:30。

为例，说明工具接口的持续演化能力；最后收束于 tone-and-style 的五条输出风范，点明“简洁、直接、无套话”的产品人格是如何被写进系统提示词底线的。

至此，‘prompt.ts’的解读全线贯通：从 §1 的软件工程任务规则，到 §2 的过早抽象与‘/help’自我披露，到 §3 的‘getActions’风险三件套，到 §4 的专用工具 vs. Bash，再到 §5 的 REPL 聚合与输出风范——Claude Code 的系统提示词把“工程纪律 + 风险意识 + 工具协议 + 输出风格”四件事统一灌进了同一个文件。它不像一份冷冰冰的技术文档，而更像一份**写给实习生的工作守则**：有规则、有态度、有文学性，也有可落地的工程细节。

6 总结与延伸

P3 从 prompt.ts 的三目运算符入口出发，自上而下走完了 getSimpleDoingTasks、getActions、getTools 以及 tone-and-style 四个 section。如果把这条阅读路线抽象成一张设计图，它的核心特征可以概括为四个字：**接口式分块**。

prompt.ts 的设计本质：接口式分块

prompt.ts 不是一个“大段独白”，而是一个**接口聚合器**。每个 section 对应一个独立的功能接口（getSimpleDoingTasks、getActions、getTools……），内部按条件分支（三目运算符、plugin hook、REPL flag）决定哪些接口被注入、哪些被隐藏。这种设计让系统提示词既有结构性，又有可扩展性。

6.1 讲师的实质性收尾：spirit and letter 与输出风范

视频末尾，讲师把镜头切到 tone-and-style 段落，逐条讲解 Claude Code 对模型输出风格的硬性约束。这些规则不是“锦上添花”，而是产品人格的底线：

- **直奔主题**——不要绕弯子，省略开场白与套话；
- **文件引用用 markdown 链接**——让用户可点击跳转；
- **GitHub issue 用专属格式**——简洁且可识别；
- **工具调用前以句号结尾**——避免悬空冒号；
- **代码与工具调用不受简洁约束**——正确性优先于 brevity。

讲师特别强调最后一条的“例外”逻辑：简洁规则不适用于代码块和工具调用。这体现了一个重要的产品设计原则——**不同内容类型适用不同质量函数**。对自然语言要求 brevity，对结构化输出要求 completeness，两者不能混为一谈。

6.2 跨章节的四条主线

回顾 §1 到 §5，四条贯穿始终的主线值得单独拎出：

1. **工程纪律线** (§1 getSimpleDoingTasks)：从“先阅读再修改”到“不要过度抽象”，系统提示词给模型划定行为底线。这些规则看似“给实习生的唠叨”，实则精准命中 LLM 编码的八大反模式。

2. **风险意识线** (§3 getActions): 以 *blast radius × reversibility × approval scope* 三件套为核心, 把”动作前的克制”写进提示词。*Spirit and letter* 与 *measure twice, cut once* 的文学性收尾, 则把规则从”枚举匹配”提升到”类推精神”。
3. **工具协议线** (§4 getTools → §5 REPL): 从”为什么不用纯 Bash”的追问, 到专用工具的四维论证(理解审阅、回滚审计、harness 可调校性、跨平台一致性), 再到 REPL 聚合把原子调用降级为虚拟消息——这条线展示了一个核心洞察: **工具设计是 agent 性能的第二杠杆**, 且与模型权重正交。
4. **产品哲学线** (§2 plugin + §5 tone-and-style): output style 不是硬编码, 而是通过 plugin + hook 动态注入; learning-output-style 把 Claude 从”代写者”变成”小老师”; tone-and-style 的五条规则构成了用户感知”Claude Code 风格简洁”的根因。三者共同指向同一个信念: **智能体的配置、反馈与风格, 都应内嵌于对话本身**。

6.3 实践启示与开放问题

从源码阅读到工程实践的五个 takeaway

1. **系统提示词是产品, 不是配置**。Anthropic 把 learning-output-style 的 README 写成产品说明书, 把 plugins 和 skills 都当作产品来做——这种视角转换决定了提示词设计的上限。
2. **把”常识”显式写进 prompt**。LLM 没有人类的”后怕”经验, 不会天然知道删除数据库和删除临时文件的区别。所有你以为”模型应该懂”的常识, 都需要被显式编码进系统提示词。
3. **专用工具 > 裸 Bash**。不是因为 Bash 不够强大, 而是因为专用工具在可逆性、可审计性、用户体验和跨平台一致性上提供了不可替代的结构化价值。
4. **Harness 与模型正交**。Can Boluk 的实验表明, 仅替换 read/edit harness 就能在 15 个开源模型上同时涨分。这意味着工具设计是一个可独立迭代的优化维度。
5. **简洁是选择, 不是默认**。tone-and-style 的规则说明, 产品人格需要通过显式约束来塑造; 模型的自然输出倾向是冗长和套话, 必须靠提示词来”修剪”。

一个开放问题: REPL 聚合的边界在哪

REPL 模式把原子工具藏进外层工具, 减轻了 LLM 的上下文负担, 但也带来一个问题: 当 REPL 脚本出错时, 模型失去了对原子工具调用的直接感知, 调试信息需要经过 REPL 工具的”二次翻译”。在什么场景下, 原子工具的显式暴露优于 REPL 聚合? 这取决于”上下文成本”与”调试透明度”之间的权衡, 也是当前 agent 架构设计中尚未完全收敛的议题。

6.4 拓展阅读

- Can Boluk – “I improved 15 LMs at Coding in one afternoon. Only the harness changed.”: 专用工具 harness 对模型性能影响的经典案例 (视频 §4 中引用)。
- Anthropic plugins-official 仓库: learning-output-style 插件的完整源码与文档, 展示了 plugin + hook 机制的具体实现 (视频 §2 中介绍)。